Loops

Introduction to Computer Programming

Chandra Gummaluru

chandra-gummaluru.github.io

Often we want to do the same thing many times. A **loop** tells Python to repeat a block of code, instead of us writing it out.

without a loop

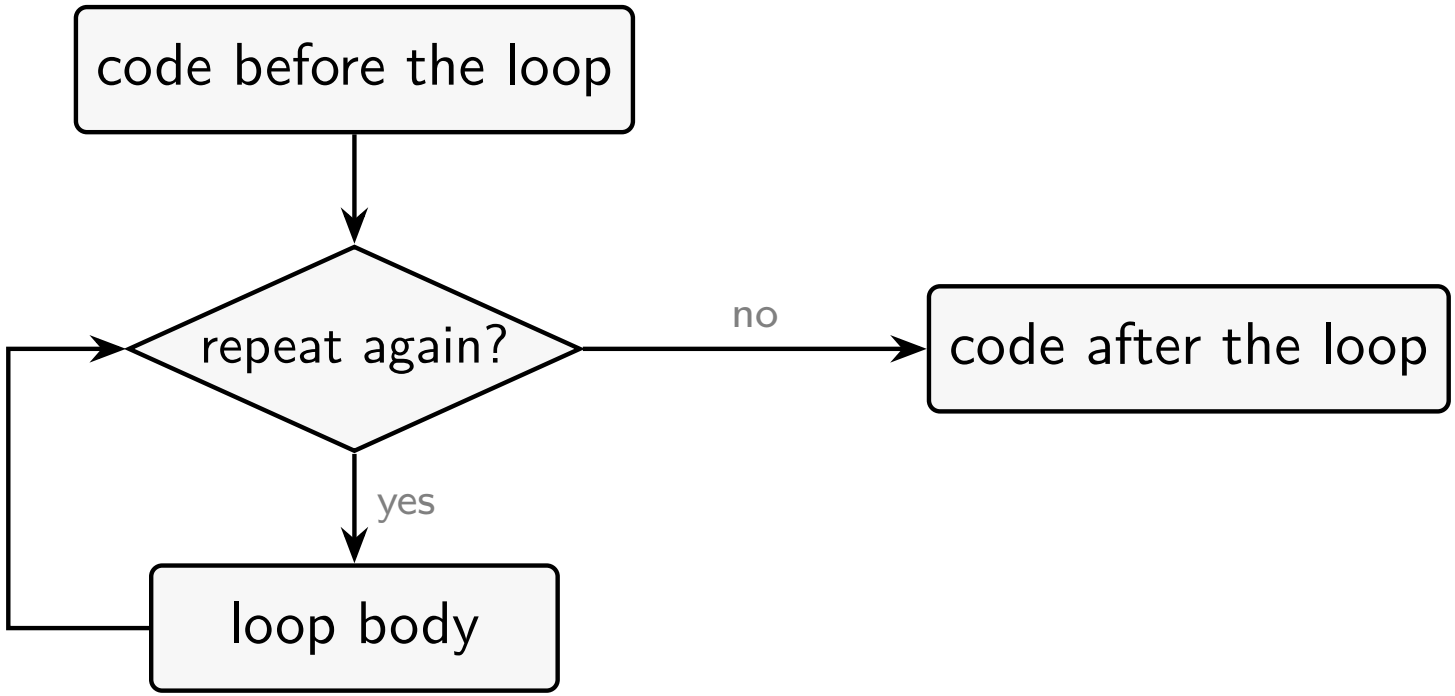
```
print(1)
print(2)
print(3)
print(4)
print(5)
```

with a loop

```
i = 1
while i <= 5:
    print(i)
    i = i + 1
```

To count to 1000, the code on the left grows to 1000 lines, while the code on the right changes one number.

Every loop has a **body**: the indented block it repeats. Each run through the body is one **pass**.



There are two types of loops. They differ in how they decide whether to repeat.

for loop

once for each element

```
s = 'hi'
for ch in s:
    print(ch)
```

stops when it runs out of elements

while loop

as long as a condition is `True`

```
x = 1
while x < 100:
    x = x * 2
```

stops when the condition is `False`

How many times does the body of this loop run?

```
1  i = 10
2  while i > 0:
3      i = i - 3
```

Answer: `4`

How many times does the following loop execute?

```
for ch in 'chandra':
    print(ch)
```

Answer: `7`

Loops are often used to build up an answer as they go, in a variable called the **accumulator**.

(1) **before the loop**

set the accumulator to an initial value

(2) **during the loop**

update the accumulator during each pass of the loop

(3) **after the loop**

the accumulator holds the answer

Depending on what you are building, the starting value for the accumulator, and how it is updated will change.

accumulator	**initial value**	**update**
string	`''`	`s_new = s_new + ch`
number	`0`/`1`	`total = total + i`
`True`/`False`	`False`	`found = True`

for loops

A **for loop** runs its body once for each element in an object. Each pass, the loop variable holds the next element.

```
s = 'cat'
for ch in s:
    print(ch)
```

ch	printed
'c'	c
'a'	a
't'	t

When there are no elements left, the loop ends.

To build a **string**, start with `''` and add to it on each pass.

```
doubled = ''
s = 'cat'
for ch in s:
    doubled = doubled + ch + ch
print(doubled)
```

ch	doubled
—	''
'c'	'cc'
'a'	'ccaa'
't'	'ccaatt'

What does this print?

```
1  s1 = 'Avatar Aang'
2  s2 = ''
3  for ch in s:
4      if ch.isupper():
5          s2 = s2 + ch.lower()
6      else:
7          s2 = s2 + ch.upper()
8  print(s2)
```

A `Avatar Aang`

B `aVATAR aANG`

C `AvAtAr AaNg`

✓ An error

vowels.py

Program

```
def only_vowels(s: str) -> str:
    """Return a string containing only the vowels from s.

    The vowels are 'a', 'e', 'i', 'o', and 'u'.
    The vowels should appear in the same order as they
    appear in s.

    Preconditions:
        s contains only lowercase letters

    Examples:
        >>> only_vowels('computer')
        'oue'
        >>> only_vowels('hello')
        'eo'
        >>> only_vowels('python')
        'o'
        >>> only_vowels('xyz')
        ''
```

```
20      """
21      vowels = ''
22      for ch in s:
23          if ch in 'aeiou':
24              vowels = vowels + ch
25      return vowels
26
27
28  print(only_vowels('computer'))
```

Shell

```
=== RUN vowels.py ===
```

```
oue
```

add_underscores.ipynb

Add at least one example to the docstring and complete the function body.

In [ ]

```
def add_underscores(s: str) -> str:
    """Return a string that contains the characters from s with an underscore
    added after every character except the last.

    >>> add_underscores('hello')
    'h_e_l_l_o'
    """
    result = ''
    for ch in s:
        result = result + ch + '_'
    return result[:-1]
```

In [2]

```
add_underscores('hello')
```

```
'h_e_l_l_o'
```

while loops

A **while loop** runs its body as long as a condition is `True`. The condition is checked at the start of every pass.

```
i = 0
while i < 3:
    print(i)
    i = i + 1
```

i	i < 3	printed
0	True	0
1	True	1
2	True	2
3	False	—

Something in the body has to move the loop toward stopping; without `i = i + 1`, it runs forever.

To build a **number**, start at `0`/`1` and add to it on each pass.

```
total = 0
i = 1
while i <= 4:
    total = total + i
    i = i + 1
print(total)
```

i	total
—	0
1	1
2	3
3	6
4	10

What does this print?

```
n = 1000
count = 0
while n > 1:
    n = n // 10
    count = count + 1
print(count)
```

Answer: `3`

digits.py

Program

```
 1  def digit_sum(n: int) -> int:
 2      """Return the sum of the digits in n.
 3
 4      Preconditions:
 5          n >= 0
 6
 7      Examples:
 8          >>> digit_sum(4096)
 9          19
10          >>> digit_sum(123)
11          6
12          >>> digit_sum(7)
13          7
14          >>> digit_sum(0)
15          0
16      """
17      total = 0
18      while n > 0:
19          total = total + n % 10
```

```
20          n = n // 10
21      return total
22
23
24  print(digit_sum(4096))
```

Shell

```
=== RUN digits.py ===
```

```
19
```

count_collatz_steps.ipynb

In math, the Collatz conjecture states that starting from any positive integer, you will eventually reach the number 1 by repeatedly applying the following two rules:

- if the number is even, divide it by 2 to get the next number in the sequence
- if the number is odd, multiply by 3 and add 1 to get the next number in the sequence

Repeatedly applying the rules generates a sequence of numbers. The Collatz step count is the number of applications of the rules required before the sequence reaches 1. For example, there are 8 Collatz steps in the Collatz sequence:

```
n = 6 -> n = 3 -> n = 10 -> n = 5 -> n = 16 -> n = 8 -> n = 4 -> n = 2 -> n = 1
```

Complete this function to count the Collatz steps for a particular number `n`.

In [ ]

```python
def count_collatz_steps(n: int) -> int:
    """Return the number of steps it takes to reach 1 by applying the two rules
    of the Collatz conjecture beginning from the positive integer n.

    Precondition: n >= 1

    >>> count_collatz_steps(6)
    8
    """
    steps = 0
    while n != 1:
        if n % 2 == 0:
            n = n // 2
        else:
            n = 3 * n + 1
        steps = steps + 1
    return steps
```

In [2]

```python
count_collatz_steps(6)
```

```
8
```

How many Collatz steps does it take to get from `7` to `1`?

Answer: `16`

To check whether a condition holds for **any** value, start at `False`, and switch to `True` as soon as you find one for which it does.

```
# is any of 1..5 a multiple of 4?
found = False
i = 1
while i <= 5:
    if i % 4 == 0:
        found = True
    i = i + 1
print(found)
```

i	i % 4	found
—	—	False
1	1	False
2	2	False
3	3	False
4	0	True
5	1	True

There is no `else`: once `found` is `True`, nothing sets it back.

9 has a divisor (3), so it should print `True`. Does it?

```
n = 9
found = False
d = 2
while d < n:
    if n % d == 0:
        found = True
    else:
        found = False
    d = d + 1
print(found)
```

A Yes

✓ No

primes.py

Program

```
def is_prime(n: int) -> bool:
    """Return whether n is a prime number.

    A prime number is a positive integer greater than 1
    that has exactly two positive divisors: 1 and itself.

    Preconditions:
        n >= 2

    Examples:
        >>> is_prime(2)
        True
        >>> is_prime(7)
        True
        >>> is_prime(9)
        False
        >>> is_prime(12)
        False
    """
```

```
20      found = False
21      d = 2
22      while d < n:
23          if n % d == 0:
24              found = True
25          d = d + 1
26      return not found
27
28
29  print(is_prime(9))
```

Shell

```
=== RUN primes.py ===
```

```
False
```

To check whether a condition holds for **every** value, start at `True` and switch to `False` as soon as you find one for which it doesn't.

```
# is every one of 1..5 odd?
all_odd = True
i = 1
while i <= 5:
    if i % 2 == 0:
        all_odd = False
    i = i + 1
print(all_odd)
```

i	i % 2	all_odd
—	—	True
1	1	True
2	0	False
3	1	False
4	0	False
5	1	False

One even number is enough to make the answer `False`; the rest can't undo it.

2 and 4 are even, so the answer should be `False`. Does this print `False`?

```
all_odd = True
i = 1
while i <= 5:
    if i % 2 == 0:
        all_odd = False
    else:
        all_odd = True
    i = i + 1
print(all_odd)
```

A Yes

✓ No

count_uppercase.ipynb

Add at least one example to the docstring and complete the function body.

In [ ]

```
def count_uppercase(s: str) -> int:
    """Return the number of uppercase letters in s.

    >>> count_uppercase('CSC108 Is Fun')
    5
    """
    count = 0
    for ch in s:
        if ch.isupper():
            count = count + 1
    return count
```

In [2]

```
count_uppercase('CSC108 Is Fun')
```

```
5
```

all_fluffy.ipynb

Add at least one example to the docstring and complete the function body.

In [ ]

```
def all_fluffy(s: str) -> bool:
    """Return True if and only if every character in s is fluffy. Fluffy
    characters are those that appear in the word 'fluffy'.

    >>> all_fluffy('fly')
    True
    >>> all_fluffy('flute')
    False
    """
    fluffy = True
    for ch in s:
        if ch not in 'fluffy':
            fluffy = False
    return fluffy
```

In [2]

```
all_fluffy('fly'), all_fluffy('flute')
```

```
(True, False)
```

A while loop can also walk through a string **by index**. Unlike a for loop, it can stop as soon as it finds what it is looking for.

```
# index of the first uppercase letter
s = 'goodMorning'
i = 0
while i < len(s) and not s[i].isupper():
    i = i + 1
print(i)
```

i	s[i]	keep going?
0	'g'	True
1	'o'	True
2	'o'	True
3	'd'	True
4	'M'	False

Checking `i < len(s)` first matters: if there is no uppercase letter, and stops before `s[i]` goes past the end of the string.

every_nth_character.ipynb

Fill in the missing code (the `____` blanks) that will make the function definition match its description.

In [ ]

```
def every_nth_character(s: str, n: int) -> str:
    """Return a string that contains every nth character from s, starting at index 0.

    Precondition: n > 0

    >>> every_nth_character('Computer Science', 3)
    'CpeSee'
    """
    result = ''
    i = 0  # The index of the next character to examine.
    while i < len(s):
        result = result + s[i]
        i = i + n
    return result
```

In [2]

```
every_nth_character('Computer Science', 3)
```

```
'CpeSee'
```

find_letter_n_times.ipynb

Fill in the missing code (the `____` blanks) that will make the function definition match its description.

In [ ]

```
def find_letter_n_times(s: str, letter: str, n: int) -> str:
    """Return the smallest substring of s starting from index 0 that contains
    n occurrences of letter.

    Precondition: letter occurs at least n times in s

    >>> find_letter_n_times('Computer Science', 'e', 2)
    'Computer Scie'
    """
    i = 0      # The index of the next character to examine.
    count = 0  # The number of occurrences of letter in s[:i].
    while count < n:
        if s[i] == letter:
            count = count + 1
        i = i + 1
    return s[:i]
```

In [2]

```
find_letter_n_times('Computer Science', 'e', 2)
```

```
'Computer Scie'
```

mystery_header.ipynb

The function below has an incomplete header and docstring. Based on the code in the function body, fill in the missing parts: the Header (including the Type Contract), Description, and Examples.

In [ ]

```
def index_of_first_digit(s: str) -> int:
    """Return the index of the first digit in s, or len(s) if s
    contains no digits.

    >>> index_of_first_digit('ab3cd7')
    2
    >>> index_of_first_digit('abc')
    3
    """
    i = 0

    while i < len(s) and s[i] not in '0123456789':
        i = i + 1
    return i
```

In [2]

```
index_of_first_digit('ab3cd7')
```

```
2
```

What does this print?

```
s = 'banana'
count = 0
for ch in s:
    if ch == 'a':
        count = count + 1
print(count)
```

Answer: `3`

Does this print `True`?

```
1  s = 'Hello'
2  all_lower = True
3  for ch in s:
4      if ch.isupper():
5          all_lower = False
6  print(all_lower)
```

A Yes

✓ No

What does this print?

```
s = 'a1b22c'
total = 0
for ch in s:
    if ch.isdigit():
        total = total + int(ch)
print(total)
```

Answer: `5`

What does this print?

```
1  s = 'computer'
2  result = ''
3  i = 0
4  while i < len(s):
5      result = result + s[i]
6      i = i + 2
7  print(result)
```

- ✓ A: cmue
- B: optr
- C: cmpt
- D: computer

What does this print?

```
n = 5
s = ''
while n > 0:
    if n % 2 == 1:
        s = s + str(n)
    n = n - 1
print(s)
```

A 135

✓ 531

C 54321

D 9

What does this print?

```
s = 'ab3cd7'
i = 0
found = False
while i < len(s) and not found:
    if s[i].isdigit():
        found = True
    i = i + 1
print(i)
```

Answer: `3`

What does this print?

```python
n = 6
bits = ''
while n > 0:
    bits = str(n % 2) + bits
    n = n // 2
print(bits)
```

A 011

✓ 110

C 6

D 0110